VORTEXBIRD

Pasión y calidad superior

15 AÑOS

www.vortexbird.com

PRUEBA TÉCNICA – DESARROLLADOR FULLSTACK

(Spring Boot + Frontend + Seguridad + Integraciones)

1. Objetivo

Evaluar el nivel técnico del candidato en:

- Diseño y arquitectura de software
- Desarrollo backend con Spring Boot
- Integración entre sistemas
- Manejo de archivos en almacenamiento tipo S3 / Blob Storage
- Desarrollo frontend moderno (Angular o React)
- Autenticación y autorización basada en roles (RBAC)
- Buenas prácticas de desarrollo y documentación técnica

El código debe ser claro, entendible y orientado a revisión técnica.

2. Contexto del Caso

La empresa requiere un sistema de gestión de **Órdenes de Pago** que permita:

- Crear órdenes
- Adjuntar una imagen o PDF de una factura
- Aprobar o rechazar órdenes
- Notificar a un sistema externo cuando una orden sea aprobada

3. Alcance y Expectativas de la Prueba

Esta prueba está diseñada para evaluar criterio técnico, claridad en el diseño y calidad del código, más que la cantidad de funcionalidades entregadas.

Se valorará especialmente:

- Estructura del proyecto
- Decisiones técnicas tomadas
- Claridad del código y comentarios
- Capacidad de priorización

4. Requerimientos Backend (Spring Boot)

4.1 APIs requeridas

- API para crear orden
- API para listar órdenes
- API para consultar órdenes por filtros
- API para obtener el detalle de una orden
- API para aprobar orden
- API para rechazar orden
- API para subir factura (multipart/form-data)

4.2 Almacenamiento tipo S3 / Blob Storage

Los archivos de factura deben almacenarse en un bucket tipo S3 o gestor de archivos equivalente.

Se evaluará:

- Separación de responsabilidades
- Uso de un servicio desacoplado (por ejemplo, `StorageService`)
- Manejo de errores asociados al almacenamiento

4.3 Integración Externa

Cuando una orden pase a estado **APPROVED**, el sistema debe notificar automáticamente a un sistema externo mediante una llamada HTTP `POST`.

Para efectos de esta prueba, se debe utilizar una **API dummy pública**, por ejemplo:

POST https://dummyjson.com/custom-response

El payload puede ser una estructura simple que represente la orden aprobada, por ejemplo:

- ID de la orden
- Estado
- Fecha de aprobación
- Usuario que aprobó

No se evaluará el contenido exacto del payload, sino la forma en que se realiza la integración.

Se valorará:

- Uso de un cliente HTTP desacoplado
- Manejo de errores HTTP (4xx / 5xx)
- Manejo de timeouts
- Registro de errores de integración

Opcional persistir la respuesta del sistema externo.
La integración puede ser simulada el flujo debe ser claro y documentado.

4.4 Seguridad Backend

El backend debe implementar:

- Autenticación basada en JWT
- Autorización por roles
- Protección de endpoints según permisos

No es necesario integrar proveedores externos de identidad (OAuth, Keycloak, etc.).
Se espera una solución simple y funcional.

5. Estados de la Orden

Las Órdenes de Pago deben manejar, como mínimo, los siguientes estados:

- PENDING
- APPROVED
- REJECTED

Las transiciones deben respetar reglas básicas de negocio (por ejemplo, una orden solo puede aprobarse o rechazarse si se encuentra en estado PENDING).

6. Requerimientos Frontend (Angular o React)

6.1 Autenticación

El sistema debe incluir una pantalla de login donde:

- El usuario ingresa email y password
- Se obtiene un JWT
- Todas las peticiones incluyan el header:

Debe manejar:

- Expiración de sesión
- Logout

6.2 Roles y Permisos (RBAC)

Roles mínimos:

- **ADMIN**
- **OPERATOR**

Permisos esperados:

ADMIN
- Listar todas las órdenes
- Filtrar órdenes
- Aprobar o rechazar órdenes
- Ver y descargar factura

OPERATOR
- Crear órdenes
- Cargar factura
- Ver detalle de orden
- No puede aprobar ni rechazar órdenes

6.3 Restricciones por Rol en Frontend

El frontend debe:

- Restringir rutas según rol
- Ocultar o deshabilitar acciones no permitidas
- Mostrar un menú dinámico dependiendo del rol

6.4 Pantallas requeridas

- Login
- Dashboard (menú dinámico según rol)
- Lista de órdenes con filtros
- Crear orden
- Detalle de orden (subir/ver factura, aprobar/rechazar según rol)

7. Manejo de Errores y Validaciones

El sistema debe manejar adecuadamente errores funcionales y técnicos.

Se valorará:

- Uso correcto de códigos HTTP
- Manejo centralizado de excepciones
- Mensajes claros y coherentes para el frontend

No se espera una solución exhaustiva, pero sí consistente.

8. Base de Datos

El candidato puede elegir la base de datos de su preferencia (por ejemplo PostgreSQL, MySQL o H2).

La elección debe documentarse en el README, explicando brevemente el motivo.

9. Auditoría de Estados (SQL)

El sistema debe garantizar que cada vez que una Orden de Pago cambie de estado, quede un rastro histórico inalterable.

Requerimiento

- Crear una tabla `order_status_log`
- Implementar un Trigger que se dispare tras un `UPDATE` en la tabla de órdenes

Debe registrar:

- ID de la orden
- Estado anterior
- Estado nuevo
- Fecha del cambio
- ID del usuario que realizó la acción

10. Mantenimiento de Datos Sensibles (SQL)

Por políticas de cumplimiento, se requiere un proceso para limpiar datos sensibles de facturas pendientes.

Requerimiento

Crear un Stored Procedure que:

- Reciba una fecha límite como parámetro
- Busque órdenes en estado `REJECTED` anteriores a esa fecha
- Marque el registro como `ARCHIVED`

El procedimiento debe manejar transacciones:

- BEGIN
- COMMIT
- ROLLBACK

11. Extras Opcionales (No obligatorios)

Los siguientes puntos **no son obligatorios**, pero suman valor a la evaluación.

11.1 Contenerización (Docker)

Opcionalmente, el candidato puede incluir:

- Dockerfile
- docker-compose.yml
- Instrucciones básicas para ejecutar el proyecto con Docker

11.2 Pruebas Unitarias

Opcionalmente, se pueden incluir pruebas unitarias para:

- Servicios
- Controladores
- Lógica de negocio relevante

No se espera una cobertura completa, sino ejemplos representativos.

11.3 Integración Externa Avanzada

Opcionalmente, se puede implementar:

- Reintentos ante fallos
- Manejo avanzado de timeouts
- Registro detallado de errores de integración

11.4 Documentación

El proyecto debe incluir un archivo `README.md` con:

- Instrucciones para ejecutar el backend
- Instrucciones para ejecutar el frontend
- Decisiones técnicas relevantes
- Funcionalidades pendientes o incompletas

Cada clase principal debe incluir un comentario inicial explicando:

- Su responsabilidad dentro del sistema
- Cómo se relaciona con otros componentes
- Por qué existe dentro de la solución

Se evaluará la claridad y utilidad de los comentarios, no la cantidad.

11. Sugerencias para el Candidato

- Prioriza una solución clara y bien estructurada sobre una extensa.
- Documenta cualquier decisión técnica relevante.
- Si algo no se alcanza a completar, explícalo.
- El código debe ser entendible por otro desarrollador sin explicación adicional.

12. Criterios de Evaluación

La prueba será evaluada considerando:

- Diseño y estructura de la solución
- Claridad y legibilidad del código
- Uso adecuado de buenas prácticas
- Manejo de seguridad, roles y estados
- Capacidad de comunicar decisiones técnicas